JavaScript is disabled on your browser.

compact1, compact2, compact3

java.time.format

Class DateTimeFormatterBuilder

java.lang.Object
    java.time.format.DateTimeFormatterBuilder

```
public final class DateTimeFormatterBuilder
extends Object
```

Builder to create date-time formatters.

This allows a `DateTimeFormatter` to be created. All date-time formatters are created ultimately using this builder.

The basic elements of date-time can all be added:

- Value - a numeric value
- Fraction - a fractional value including the decimal place. Always use this when outputting fractions to ensure that the fraction is parsed correctly
- Text - the textual equivalent for the value
- OffsetId/Offset - the zone offset
- ZoneId - the time-zone id
- ZoneText - the name of the time-zone
- ChronologyId - the chronology id
- ChronologyText - the name of the chronology
- Literal - a text literal
- Nested and Optional - formats can be nested or made optional

In addition, any of the elements may be decorated by padding, either with spaces or any other character.

Finally, a shorthand pattern, mostly compatible with `java.text.SimpleDateFormat SimpleDateFormat` can be used, see `appendPattern(String)`. In practice, this simply parses the pattern and calls other methods on the builder.

Implementation Requirements:

This class is a mutable builder intended for use from a single thread.

Since:

1.8

Constructor Summary

Constructors

Constructor	Description
`DateTimeFormatterBuilder()`	Constructs a new instance of the builder.

Method Summary

Concrete Methods

Modifier and Type	Method	Desc
`DateTimeFormatterBuilder`	`append(DateTimeFormatter formatter)`	App elen form buil
`DateTimeFormatterBuilder`	`appendChronologyId()`	App chro as 'I 'Tha the
`DateTimeFormatterBuilder`	`appendChronologyText(TextStyle textStyle)`	App chro the
`DateTimeFormatterBuilder`	`appendFraction(TemporalField field, int minWidth, int maxWidth, boolean decimalPoint)`	App frac date the
`DateTimeFormatterBuilder`	`appendInstant()`	App usin the form frac grou
`DateTimeFormatterBuilder`	`appendInstant(int fractionalDigits)`	App usin

		the cont num frac
DateTimeFormatterBuilder	appendLiteral(char literal)	App cha the
DateTimeFormatterBuilder	appendLiteral(String literal)	App liter form
DateTimeFormatterBuilder	appendLocalized(FormatStyle dateStyle, FormatStyle timeStyle)	App date the
DateTimeFormatterBuilder	appendLocalizedOffset(TextStyle style)	App loca offs 'GM the
DateTimeFormatterBuilder	appendOffset(String pattern, String noOffsetText)	App offs '+0 form
DateTimeFormatterBuilder	appendOffsetId()	App offs '+0 form
DateTimeFormatterBuilder	appendOptional(DateTimeFormatter formatter)	App form buil opti pars
DateTimeFormatterBuilder	appendPattern(String pattern)	App elen the patt buil
DateTimeFormatterBuilder	appendText(TemporalField field)	App a da the the

DateTimeFormatterBuilder	appendText(TemporalField field, Map<Long,String> textLookup)	App a da the the to s
DateTimeFormatterBuilder	appendText(TemporalField field, TextStyle textStyle)	App a da the
DateTimeFormatterBuilder	appendValue(TemporalField field)	App of a to t usin outp
DateTimeFormatterBuilder	appendValue(TemporalField field, int width)	App of a to t usin zero app
DateTimeFormatterBuilder	appendValue(TemporalField field, int minWidth, int maxWidth, SignStyle signStyle)	App of a to t prov cont form
DateTimeFormatterBuilder	appendValueReduced(TemporalField field, int width, int maxWidth, ChronoLocalDate baseDate)	App redu date the
DateTimeFormatterBuilder	appendValueReduced(TemporalField field, int width, int maxWidth, int baseValue)	App redu date the
DateTimeFormatterBuilder	appendZoneId()	App zon 'Eur '+0 form
DateTimeFormatterBuilder	appendZoneOrOffsetId()	App zon 'Eur

		'+0 form best ID.
DateTimeFormatterBuilder	**appendZoneRegionId**()	App zon such Pari form the zon Zon
DateTimeFormatterBuilder	**appendZoneText**(**TextStyle** textStyle)	App zon 'Bri Tim form
DateTimeFormatterBuilder	**appendZoneText**(**TextStyle** textStyle, **Set**<**ZoneId**> preferredZones)	App zon 'Bri Tim form
static **String**	**getLocalizedDateTimePattern**(**FormatStyle** dateStyle, **FormatStyle** timeStyle, **Chronology** chrono, **Locale** locale)	Gets patt and a lo chro
DateTimeFormatterBuilder	**optionalEnd**()	End sect
DateTimeFormatterBuilder	**optionalStart**()	Mar opti
DateTimeFormatterBuilder	**padNext**(int padWidth)	Cau add pars fixe spa
DateTimeFormatterBuilder	**padNext**(int padWidth, char padChar)	Cau add pars fixe
DateTimeFormatterBuilder	**parseCaseInsensitive**()	

		Cha styl inse rem form
DateTimeFormatterBuilder	parseCaseSensitive()	Cha styl sens rem form
DateTimeFormatterBuilder	parseDefaulting(TemporalField field, long value)	App valu the use
DateTimeFormatterBuilder	parseLenient()	Cha styl for of th
DateTimeFormatterBuilder	parseStrict()	Cha styl the the
DateTimeFormatter	toFormatter()	Com buil the Dat usin loca
DateTimeFormatter	toFormatter(Locale locale)	Com buil the Dat usin loca

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Detail

DateTimeFormatterBuilder

```
public DateTimeFormatterBuilder()
```

Constructs a new instance of the builder.

Method Detail

getLocalizedDateTimePattern

```
public static String getLocalizedDateTimePattern(FormatStyle dateStyle,
                                                 FormatStyle timeStyle,
                                                 Chronology chrono,
                                                 Locale locale)
```

Gets the formatting pattern for date and time styles for a locale and chronology. The locale and chronology are used to lookup the locale specific format for the requested dateStyle and/or timeStyle.

Parameters:

`dateStyle - the FormatStyle for the date, null for time-only pattern`

`timeStyle - the FormatStyle for the time, null for date-only pattern`

`chrono - the Chronology, non-null`

`locale - the locale, non-null`

Returns:

`the locale and Chronology specific formatting pattern`

Throws:

`IllegalArgumentException - if both dateStyle and timeStyle are null`

parseCaseSensitive

```
public DateTimeFormatterBuilder parseCaseSensitive()
```

Changes the parse style to be case sensitive for the remainder of the formatter.

Parsing can be case sensitive or insensitive - by default it is case sensitive. This method allows the case sensitivity setting of parsing to be changed.

Calling this method changes the state of the builder such that all subsequent builder method calls will parse text in case sensitive mode. See `parseCaseInsensitive()` for the opposite setting. The parse case sensitive/ insensitive methods may be called at any point in the builder, thus the parser can swap between case parsing modes multiple times during the parse.

Since the default is case sensitive, this method should only be used after a previous call to `#parseCaseInsensitive`.

Returns:

`this, for chaining, not null`

parseCaseInsensitive

```
public DateTimeFormatterBuilder parseCaseInsensitive()
```

Changes the parse style to be case insensitive for the remainder of the formatter.

Parsing can be case sensitive or insensitive - by default it is case sensitive. This method allows the case sensitivity setting of parsing to be changed.

Calling this method changes the state of the builder such that all subsequent builder method calls will parse text in case insensitive mode. See `parseCaseSensitive()` for the opposite setting. The parse case sensitive/ insensitive methods may be called at any point in the builder, thus the parser can swap between case parsing modes multiple times during the parse.

Returns:

`this, for chaining, not null`

parseStrict

```
public DateTimeFormatterBuilder parseStrict()
```

Changes the parse style to be strict for the remainder of the formatter.

Parsing can be strict or lenient - by default its strict. This controls the degree of flexibility in matching the text and sign styles.

When used, this method changes the parsing to be strict from this point onwards. As strict is the default, this is normally only needed after calling `parseLenient()`. The change will remain in force until the end of the formatter that is eventually constructed or until `parseLenient` is called.

Returns:

`this, for chaining, not null`

parseLenient

```
public DateTimeFormatterBuilder parseLenient()
```

Changes the parse style to be lenient for the remainder of the formatter. Note that case sensitivity is set separately to this method.

Parsing can be strict or lenient - by default its strict. This controls the degree of flexibility in matching the text and sign styles. Applications calling this method should typically also call `parseCaseInsensitive()`.

When used, this method changes the parsing to be lenient from this point onwards. The change will remain in force until the end of the formatter that is eventually constructed or until `parseStrict` is called.

Returns:

`this, for chaining, not null`

parseDefaulting

```
public DateTimeFormatterBuilder parseDefaulting(TemporalField field,
                                                long value)
```

Appends a default value for a field to the formatter for use in parsing.

This appends an instruction to the builder to inject a default value into the parsed result. This is especially useful in conjunction with optional parts of the formatter.

For example, consider a formatter that parses the year, followed by an optional month, with a further optional day-of-month. Using such a formatter would require the calling code to check whether a full date, year-month or just a year had been parsed. This method can be used to default the month and day-of-month to a sensible value, such as the first of the month, allowing the calling code to always get a date.

During formatting, this method has no effect.

During parsing, the current state of the parse is inspected. If the specified field has no associated value, because it has not been parsed successfully at that point, then the specified value is injected into the parse result. Injection is immediate, thus the field-value pair will be visible to any subsequent elements in the formatter. As such, this method is normally called at the end of the builder.

Parameters:

`field - the field to default the value of, not null`

`value - the value to default the field to`

Returns:

`this, for chaining, not null`

appendValue

```
public DateTimeFormatterBuilder appendValue(TemporalField field)
```

Appends the value of a date-time field to the formatter using a normal output style.

The value of the field will be output during a format. If the value cannot be obtained then an exception will be thrown.

The value will be printed as per the normal format of an integer value. Only negative numbers will be signed. No padding will be added.

The parser for a variable width value such as this normally behaves greedily, requiring one digit, but accepting as many digits as possible. This behavior can be affected by 'adjacent value parsing'. See `appendValue(java.time.temporal.TemporalField, int)` for full details.

Parameters:

`field - the field to append, not null`

Returns:

`this, for chaining, not null`

appendValue

```
public DateTimeFormatterBuilder appendValue(TemporalField field,
                                            int width)
```

Appends the value of a date-time field to the formatter using a fixed width, zero-padded approach.

The value of the field will be output during a format. If the value cannot be obtained then an exception will be thrown.

The value will be zero-padded on the left. If the size of the value means that it cannot be printed within the width then an exception is thrown. If the value of the field is negative then an exception is thrown during formatting.

This method supports a special technique of parsing known as 'adjacent value parsing'. This technique solves the problem where a value, variable or fixed width, is followed by one or more fixed length values. The standard parser is greedy, and thus it would normally steal the digits that are needed by the fixed width value parsers that follow the variable width one.

No action is required to initiate 'adjacent value parsing'. When a call to `appendValue` is made, the builder enters adjacent value parsing setup mode. If the immediately subsequent method call or calls on the same builder are for a fixed width value, then the parser will reserve space so that the fixed width values can be parsed.

For example, consider `builder.appendValue(YEAR).appendValue(MONTH_OF_YEAR, 2);` The year is a variable width parse of between 1 and 19 digits. The month is a fixed width parse of 2 digits. Because these were appended to the same builder immediately after one another, the year parser will reserve two digits for the month to parse. Thus, the text '201106' will correctly parse to a year of 2011 and a month of 6. Without adjacent value parsing, the year would greedily parse all six digits and leave nothing for the month.

Adjacent value parsing applies to each set of fixed width not-negative values in the parser that immediately follow any kind of value, variable or fixed width. Calling any other append method will end the setup of adjacent value parsing. Thus, in the unlikely event that you need to avoid adjacent value parsing behavior, simply add the `appendValue` to another `DateTimeFormatterBuilder` and add that to this builder.

If adjacent parsing is active, then parsing must match exactly the specified number of digits in both strict and lenient modes. In addition, no positive or negative sign is permitted.

Parameters:

`field - the field to append, not null`

`width - the width of the printed field, from 1 to 19`

Returns:

`this, for chaining, not null`

Throws:

`IllegalArgumentException - if the width is invalid`

appendValue

```
public DateTimeFormatterBuilder appendValue(TemporalField field,
                                            int minWidth,
                                            int maxWidth,
                                            SignStyle signStyle)
```

Appends the value of a date-time field to the formatter providing full control over formatting.

The value of the field will be output during a format. If the value cannot be obtained then an exception will be thrown.

This method provides full control of the numeric formatting, including zero-padding and the positive/negative sign.

The parser for a variable width value such as this normally behaves greedily, accepting as many digits as possible. This behavior can be affected by 'adjacent value parsing'. See `appendValue(java.time.temporal.TemporalField, int)` for full details.

In strict parsing mode, the minimum number of parsed digits is `minWidth` and the maximum is `maxWidth`. In lenient parsing mode, the minimum number of parsed digits is one and the maximum is 19 (except as limited by adjacent value parsing).

If this method is invoked with equal minimum and maximum widths and a sign style of `NOT_NEGATIVE` then it delegates to `appendValue(TemporalField,int)`. In this scenario, the formatting and parsing behavior described there occur.

Parameters:

`field - the field to append, not null`

`minWidth - the minimum field width of the printed field, from 1 to 19`

`maxWidth - the maximum field width of the printed field, from 1 to 19`

`signStyle - the positive/negative output style, not null`

Returns:

`this, for chaining, not null`

Throws:

`IllegalArgumentException - if the widths are invalid`

appendValueReduced

```
public DateTimeFormatterBuilder appendValueReduced(TemporalField field,
                                                   int width,
                                                   int maxWidth,
                                                   int baseValue)
```

Appends the reduced value of a date-time field to the formatter.

Since fields such as year vary by chronology, it is recommended to use the `appendValueReduced(TemporalField, int, int, ChronoLocalDate)` date} variant of this method in most cases. This variant is suitable for simple fields or working with only the ISO chronology.

For formatting, the `width` and `maxWidth` are used to determine the number of characters to format. If they are equal then the format is fixed width. If the value of the field is within the range of the `baseValue` using `width` characters

then the reduced value is formatted otherwise the value is truncated to fit `maxWidth`. The rightmost characters are output to match the width, left padding with zero.

For strict parsing, the number of characters allowed by `width` to `maxWidth` are parsed. For lenient parsing, the number of characters must be at least 1 and less than 10. If the number of digits parsed is equal to `width` and the value is positive, the value of the field is computed to be the first number greater than or equal to the `baseValue` with the same least significant characters, otherwise the value parsed is the field value. This allows a reduced value to be entered for values in range of the baseValue and width and absolute values can be entered for values outside the range.

For example, a base value of `1980` and a width of `2` will have valid values from `1980` to `2079`. During parsing, the text `"12"` will result in the value `2012` as that is the value within the range where the last two characters are "12". By contrast, parsing the text `"1915"` will result in the value `1915`.

Parameters:

`field - the field to append, not null`

`width - the field width of the printed and parsed field, from 1 to 10`

`maxWidth - the maximum field width of the printed field, from 1 to 10`

`baseValue - the base value of the range of valid values`

Returns:

`this, for chaining, not null`

Throws:

`IllegalArgumentException - if the width or base value is invalid`

appendValueReduced

```
public DateTimeFormatterBuilder appendValueReduced(TemporalField field,
                                                   int width,
                                                   int maxWidth,
                                                   ChronoLocalDate baseDate)
```

Appends the reduced value of a date-time field to the formatter.

This is typically used for formatting and parsing a two digit year.

The base date is used to calculate the full value during parsing. For example, if the base date is 1950-01-01 then parsed values for a two digit year parse will be in the range 1950-01-01 to 2049-12-31. Only the year would be extracted from the date, thus a base date of 1950-08-25 would also parse to the range 1950-01-01 to 2049-12-31. This behavior is necessary to support

fields such as week-based-year or other calendar systems where the parsed value does not align with standard ISO years.

The exact behavior is as follows. Parse the full set of fields and determine the effective chronology using the last chronology if it appears more than once. Then convert the base date to the effective chronology. Then extract the specified field from the chronology-specific base date and use it to determine the `baseValue` used below.

For formatting, the `width` and `maxWidth` are used to determine the number of characters to format. If they are equal then the format is fixed width. If the value of the field is within the range of the `baseValue` using `width` characters then the reduced value is formatted otherwise the value is truncated to fit `maxWidth`. The rightmost characters are output to match the width, left padding with zero.

For strict parsing, the number of characters allowed by `width` to `maxWidth` are parsed. For lenient parsing, the number of characters must be at least 1 and less than 10. If the number of digits parsed is equal to `width` and the value is positive, the value of the field is computed to be the first number greater than or equal to the `baseValue` with the same least significant characters, otherwise the value parsed is the field value. This allows a reduced value to be entered for values in range of the baseValue and width and absolute values can be entered for values outside the range.

For example, a base value of `1980` and a width of `2` will have valid values from `1980` to `2079`. During parsing, the text `"12"` will result in the value `2012` as that is the value within the range where the last two characters are "12". By contrast, parsing the text `"1915"` will result in the value `1915`.

Parameters:

`field - the field to append, not null`

`width - the field width of the printed and parsed field, from 1 to 10`

`maxWidth - the maximum field width of the printed field, from 1 to 10`

`baseDate - the base date used to calculate the base value for the range of valid values in the parsed chronology, not null`

Returns:

`this, for chaining, not null`

Throws:

`IllegalArgumentException - if the width or base value is invalid`

appendFraction

```
public DateTimeFormatterBuilder appendFraction(TemporalField field,
                                               int minWidth,
                                               int maxWidth,
                                               boolean decimalPoint)
```

Appends the fractional value of a date-time field to the formatter.

The fractional value of the field will be output including the preceding decimal point. The preceding value is not output. For example, the second-of-minute value of 15 would be output as `.25`.

The width of the printed fraction can be controlled. Setting the minimum width to zero will cause no output to be generated. The printed fraction will have the minimum width necessary between the minimum and maximum widths - trailing zeroes are omitted. No rounding occurs due to the maximum width - digits are simply dropped.

When parsing in strict mode, the number of parsed digits must be between the minimum and maximum width. When parsing in lenient mode, the minimum width is considered to be zero and the maximum is nine.

If the value cannot be obtained then an exception will be thrown. If the value is negative an exception will be thrown. If the field does not have a fixed set of valid values then an exception will be thrown. If the field value in the date-time to be printed is invalid it cannot be printed and an exception will be thrown.

Parameters:

`field - the field to append, not null`

`minWidth - the minimum width of the field excluding the decimal point, from 0 to 9`

`maxWidth - the maximum width of the field excluding the decimal point, from 1 to 9`

`decimalPoint - whether to output the localized decimal point symbol`

Returns:

`this, for chaining, not null`

Throws:

`IllegalArgumentException - if the field has a variable set of valid values or either width is invalid`

appendText

```
public DateTimeFormatterBuilder appendText(TemporalField field)
```

Appends
the text of a date-time field to the formatter using the full text style.

The text of the field will be output during a format. The value must be within the valid range of the field. If the value cannot be obtained then an exception will be thrown. If the field has no textual representation, then the numeric value will be used.

The value will be printed as per the normal format of an integer value. Only negative numbers will be signed. No padding will be added.

Parameters:

`field - the field to append, not null`

Returns:

`this, for chaining, not null`

appendText

```
public DateTimeFormatterBuilder appendText(TemporalField field,
                                           TextStyle textStyle)
```

Appends the text of a date-time field to the formatter.

The text of the field will be output during a format. The value must be within the valid range of the field. If the value cannot be obtained then an exception will be thrown. If the field has no textual representation, then the numeric value will be used.

The value will be printed as per the normal format of an integer value. Only negative numbers will be signed. No padding will be added.

Parameters:

`field - the field to append, not null`

`textStyle - the text style to use, not null`

Returns:

`this, for chaining, not null`

appendText

```
public DateTimeFormatterBuilder appendText(TemporalField field,
                                           Map<Long,String> textLookup)
```

Appends the text of a date-time field to the formatter using the specified map to supply the text.

The standard text outputting methods use the localized text in the JDK. This method allows that text to be specified directly. The supplied map is not validated by the builder to ensure that formatting or parsing is possible, thus an invalid map may throw an error during later use.

Supplying the map of text provides considerable flexibility in formatting and parsing. For example, a legacy application might require or supply the months of the year as "JNY", "FBY", "MCH" etc. These do not match the standard set of text for localized month names. Using this method, a map can be created which defines the connection between each value and the text:

```
Map<Long, String> map = new HashMap<>();
map.put(1L, "JNY");
map.put(2L, "FBY");
map.put(3L, "MCH");
...
builder.appendText(MONTH_OF_YEAR, map);
```

Other uses might be to output the value with a suffix, such as "1st", "2nd", "3rd", or as Roman numerals "I", "II", "III", "IV".

During formatting, the value is obtained and checked that it is in the valid range. If text is not available for the value then it is output as a number. During parsing, the parser will match against the map of text and numeric values.

Parameters:

`field - the field to append, not null`

`textLookup - the map from the value to the text`

Returns:

`this, for chaining, not null`

appendInstant

```
public DateTimeFormatterBuilder appendInstant()
```

Appends an instant using ISO-8601 to the formatter, formatting fractional digits in groups of three.

Instants have a fixed output format. They are converted to a date-time with a zone-offset of UTC and formatted using the standard ISO-8601 format. With this method, formatting nano-of-second outputs zero, three, six or nine digits digits as necessary. The localized decimal style is not used.

The instant is obtained using INSTANT_SECONDS and optionally (@code NANO_OF_SECOND). The value of INSTANT_SECONDS may be outside the maximum range of LocalDateTime.

The resolver style has no effect on instant parsing. The end-of-day time of '24:00' is handled as midnight at the start of the following day. The leap-second time of '23:59:59' is handled to some degree, see DateTimeFormatter.parsedLeapSecond() for full details.

An alternative to this method is to format/parse the instant as a single epoch-seconds value. That is achieved using appendValue(INSTANT_SECONDS).

Returns:

this, for chaining, not null

appendInstant

```
public DateTimeFormatterBuilder appendInstant(int fractionalDigits)
```

Appends an instant using ISO-8601 to the formatter with control over the number of fractional digits.

Instants have a fixed output format, although this method provides some control over the fractional digits. They are converted to a date-time with a zone-offset of UTC and printed using the standard ISO-8601 format. The localized decimal style is not used.

The fractionalDigits parameter allows the output of the fractional second to be controlled. Specifying zero will cause no fractional digits to be output. From 1 to 9 will output an increasing number of digits, using zero right-padding if necessary. The special value -1 is used to output as many digits as necessary to avoid any trailing zeroes.

When parsing in strict mode, the number of parsed digits must match the fractional digits. When parsing in lenient mode, any number of fractional digits from zero to nine are accepted.

The instant is obtained using INSTANT_SECONDS and optionally (@code NANO_OF_SECOND). The value of INSTANT_SECONDS may be outside the maximum range of LocalDateTime.

The resolver style has no effect on instant parsing. The end-of-day time of '24:00' is handled as midnight at the start of the following day. The leap-second time of '23:59:60' is handled to some degree, see DateTimeFormatter.parsedLeapSecond() for full details.

An alternative to this method is to format/parse the instant as a single epoch-seconds value. That is achieved using appendValue(INSTANT_SECONDS).

Parameters:

`fractionalDigits - the number of fractional second digits to format with, from 0 to 9, or -1 to use as many digits as necessary`

Returns:

`this, for chaining, not null`

appendOffsetId

```
public DateTimeFormatterBuilder appendOffsetId()
```

Appends the zone offset, such as '+01:00', to the formatter.

This appends an instruction to format/parse the offset ID to the builder. This is equivalent to calling `appendOffset("+HH:MM:ss", "Z")`.

Returns:

`this, for chaining, not null`

appendOffset

```
public DateTimeFormatterBuilder appendOffset(String pattern,
                                             String noOffsetText)
```

Appends the zone offset, such as '+01:00', to the formatter.

This appends an instruction to format/parse the offset ID to the builder.

During formatting, the offset is obtained using a mechanism equivalent to querying the temporal with `TemporalQueries.offset()`. It will be printed using the format defined below. If the offset cannot be obtained then an exception is thrown unless the section of the formatter is optional.

During parsing, the offset is parsed using the format defined below. If the offset cannot be parsed then an exception is thrown unless the section of the formatter is optional.

The format of the offset is controlled by a pattern which must be one of the following:

- `+HH` - hour only, ignoring minute and second
- `+HHmm` - hour, with minute if non-zero, ignoring second, no colon
- `+HH:mm` - hour, with minute if non-zero, ignoring second, with colon
- `+HHMM` - hour and minute, ignoring second, no colon
- `+HH:MM` - hour and minute, ignoring second, with colon
- `+HHMMss` - hour and minute, with second if non-zero, no colon
- `+HH:MM:ss` - hour and minute, with second if non-zero, with colon
- `+HHMMSS` - hour, minute and second, no colon
- `+HH:MM:SS` - hour, minute and second, with colon

The "no offset" text controls what text is printed when the total amount of the offset fields to be output is zero. Example values would be 'Z', '+00:00', 'UTC' or 'GMT'. Three formats are accepted for parsing UTC - the "no offset" text, and the plus and minus versions of zero defined by the pattern.

Parameters:

`pattern - the pattern to use, not null`

`noOffsetText - the text to use when the offset is zero, not null`

Returns:

`this, for chaining, not null`

appendLocalizedOffset

```
public DateTimeFormatterBuilder appendLocalizedOffset(TextStyle style)
```

Appends the localized zone offset, such as 'GMT+01:00', to the formatter.

This appends a localized zone offset to the builder, the format of the localized offset is controlled by the specified `style` to this method:

- `full` - formats with localized offset text, such as 'GMT, 2-digit hour and minute field, optional second field if non-zero, and colon.
- `short` - formats with localized offset text, such as 'GMT, hour without leading zero, optional 2-digit minute and second if non-zero, and colon.

During formatting, the offset is obtained using a mechanism equivalent to querying the temporal with `TemporalQueries.offset()`. If the offset cannot be obtained then an exception is thrown unless the section of the formatter is optional.

During parsing, the offset is parsed using the format defined above. If the offset cannot be parsed then an exception is thrown unless the section of the formatter is optional.

Parameters:

`style - the format style to use, not null`

Returns:

`this, for chaining, not null`

Throws:

`IllegalArgumentException - if style is neither full nor short`

appendZoneId

```
public DateTimeFormatterBuilder appendZoneId()
```

Appends the time-zone ID, such as 'Europe/Paris' or '+02:00', to the formatter.

This appends an instruction to format/parse the zone ID to the builder. The zone ID is obtained in a strict manner suitable for `ZonedDateTime`. By contrast, `OffsetDateTime` does not have a zone ID suitable for use with this method, see `appendZoneOrOffsetId()`.

During formatting, the zone is obtained using a mechanism equivalent to querying the temporal with `TemporalQueries.zoneId()`. It will be printed using the result of `ZoneId.getId()`. If the zone cannot be obtained then an exception is thrown unless the section of the formatter is optional.

During parsing, the text must match a known zone or offset. There are two types of zone ID, offset-based, such as '+01:30' and region-based, such as 'Europe/London'. These are parsed differently. If the parse starts with '+', '-', 'UT', 'UTC' or 'GMT', then the parser expects an offset-based zone and will not match region-based zones. The offset ID, such as '+02:30', may be at the start of the parse, or prefixed by 'UT', 'UTC' or 'GMT'. The offset ID parsing is equivalent to using `appendOffset(String, String)` using the arguments 'HH:MM:ss' and the no offset string '0'. If the parse starts with 'UT', 'UTC' or 'GMT', and the parser cannot match a following offset ID, then `ZoneOffset.UTC` is selected. In all other cases, the list of known region-based zones is used to find the longest available match. If no match is found, and the parse starts with 'Z', then `ZoneOffset.UTC` is selected. The parser uses the case sensitive setting.

For example, the following will parse:

```
"Europe/London"           -- ZoneId.of("Europe/London")
"Z"                       -- ZoneOffset.UTC
"UT"                      -- ZoneId.of("UT")
"UTC"                     -- ZoneId.of("UTC")
"GMT"                     -- ZoneId.of("GMT")
"+01:30"                  -- ZoneOffset.of("+01:30")
"UT+01:30"                -- ZoneOffset.of("+01:30")
"UTC+01:30"               -- ZoneOffset.of("+01:30")
"GMT+01:30"               -- ZoneOffset.of("+01:30")
```

Returns:

`this, for chaining, not null`

See Also:

`appendZoneRegionId()`

appendZoneRegionId

```
public DateTimeFormatterBuilder appendZoneRegionId()
```

Appends the time-zone region ID, such as 'Europe/Paris', to the formatter, rejecting the zone ID if it is a `ZoneOffset`.

This appends an instruction to format/parse the zone ID to the builder only if it is a region-based ID.

During formatting, the zone is obtained using a mechanism equivalent to querying the temporal with `TemporalQueries.zoneId()`. If the zone is a `ZoneOffset` or it cannot be obtained then an exception is thrown unless the section of the formatter is optional. If the zone is not an offset, then the zone will be printed using the zone ID from `ZoneId.getId()`.

During parsing, the text must match a known zone or offset. There are two types of zone ID, offset-based, such as '+01:30' and region-based, such as 'Europe/London'. These are parsed differently. If the parse starts with '+', '-', 'UT', 'UTC' or 'GMT', then the parser expects an offset-based zone and will not match region-based zones. The offset ID, such as '+02:30', may be at the start of the parse, or prefixed by 'UT', 'UTC' or 'GMT'. The offset ID parsing is equivalent to using `appendOffset(String, String)` using the arguments 'HH:MM:ss' and the no offset string '0'. If the parse starts with 'UT', 'UTC' or 'GMT', and the parser cannot match a following offset ID, then `ZoneOffset.UTC` is selected. In all other cases, the list of known region-based zones is used to find the longest available match. If no match is found, and the parse starts with 'Z', then `ZoneOffset.UTC` is selected. The parser uses the case sensitive setting.

For example, the following will parse:

```
"Europe/London"           -- ZoneId.of("Europe/London")
"Z"                       -- ZoneOffset.UTC
"UT"                      -- ZoneId.of("UT")
"UTC"                     -- ZoneId.of("UTC")
"GMT"                     -- ZoneId.of("GMT")
"+01:30"                  -- ZoneOffset.of("+01:30")
"UT+01:30"                -- ZoneOffset.of("+01:30")
"UTC+01:30"               -- ZoneOffset.of("+01:30")
"GMT+01:30"               -- ZoneOffset.of("+01:30")
```

Note that this method is identical to `appendZoneId()` except in the mechanism used to obtain the zone. Note also that parsing accepts offsets, whereas formatting will never produce one.

Returns:

`this, for chaining, not null`

See Also:

`appendZoneId()`

appendZoneOrOffsetId

```
public DateTimeFormatterBuilder appendZoneOrOffsetId()
```

Appends the time-zone ID, such as 'Europe/Paris' or '+02:00', to the formatter, using the best available zone ID.

This appends an instruction to format/parse the best available zone or offset ID to the builder. The zone ID is obtained in a lenient manner that first attempts to find a true zone ID, such as that on `ZonedDateTime`, and then attempts to find an offset, such as that on `OffsetDateTime`.

During formatting, the zone is obtained using a mechanism equivalent to querying the temporal with `TemporalQueries.zone()`. It will be printed using the result of `ZoneId.getId()`. If the zone cannot be obtained then an exception is thrown unless the section of the formatter is optional.

During parsing, the text must match a known zone or offset. There are two types of zone ID, offset-based, such as '+01:30' and region-based, such as 'Europe/London'. These are parsed differently. If the parse starts with '+', '-', 'UT', 'UTC' or 'GMT', then the parser expects an offset-based zone and will not match region-based zones. The offset ID, such as '+02:30', may be at the start of the parse, or prefixed by 'UT', 'UTC' or 'GMT'. The offset ID parsing is equivalent to using `appendOffset(String, String)` using the arguments 'HH:MM:ss' and the no offset string '0'. If the parse starts with 'UT', 'UTC' or 'GMT', and the parser cannot match a following offset ID, then `ZoneOffset.UTC` is selected. In all other cases, the list of known region-based zones is used to find the longest available match. If no match is found, and the parse starts with 'Z', then `ZoneOffset.UTC` is selected. The parser uses the case sensitive setting.

For example, the following will parse:

```
"Europe/London"          -- ZoneId.of("Europe/London")
"Z"                      -- ZoneOffset.UTC
"UT"                     -- ZoneId.of("UT")
"UTC"                    -- ZoneId.of("UTC")
"GMT"                    -- ZoneId.of("GMT")
"+01:30"                 -- ZoneOffset.of("+01:30")
"UT+01:30"               -- ZoneOffset.of("UT+01:30")
"UTC+01:30"              -- ZoneOffset.of("UTC+01:30")
"GMT+01:30"              -- ZoneOffset.of("GMT+01:30")
```

Note that this method is identical to `appendZoneId()` except in the mechanism used to obtain the zone.

Returns:

`this, for chaining, not null`

See Also:

appendZoneId()

appendZoneText

```
public DateTimeFormatterBuilder appendZoneText(TextStyle textStyle)
```

Appends the time-zone name, such as 'British Summer Time', to the formatter.

This appends an instruction to format/parse the textual name of the zone to the builder.

During formatting, the zone is obtained using a mechanism equivalent to querying the temporal with TemporalQueries.zoneId(). If the zone is a ZoneOffset it will be printed using the result of ZoneOffset.getId(). If the zone is not an offset, the textual name will be looked up for the locale set in the DateTimeFormatter. If the temporal object being printed represents an instant, then the text will be the summer or winter time text as appropriate. If the lookup for text does not find any suitable result, then the ID will be printed instead. If the zone cannot be obtained then an exception is thrown unless the section of the formatter is optional.

During parsing, either the textual zone name, the zone ID or the offset is accepted. Many textual zone names are not unique, such as CST can be for both "Central Standard Time" and "China Standard Time". In this situation, the zone id will be determined by the region information from formatter's locale and the standard zone id for that area, for example, America/New_York for the America Eastern zone. The appendZoneText(TextStyle, Set) may be used to specify a set of preferred ZoneId in this situation.

Parameters:

`textStyle - the text style to use, not null`

Returns:

`this, for chaining, not null`

appendZoneText

```
public DateTimeFormatterBuilder appendZoneText(TextStyle textStyle,
                                               Set<ZoneId> preferredZones)
```

Appends the time-zone name, such as 'British Summer Time', to the formatter.

This appends an instruction to format/parse the textual name of the zone to the builder.

During formatting, the zone is obtained using a mechanism equivalent to querying the temporal with TemporalQueries.zoneId(). If the zone is a

ZoneOffset it will be printed using the result of ZoneOffset.getId(). If the zone is not an offset, the textual name will be looked up for the locale set in the DateTimeFormatter. If the temporal object being printed represents an instant, then the text will be the summer or winter time text as appropriate. If the lookup for text does not find any suitable result, then the ID will be printed instead. If the zone cannot be obtained then an exception is thrown unless the section of the formatter is optional.

During parsing, either the textual zone name, the zone ID or the offset is accepted. Many textual zone names are not unique, such as CST can be for both "Central Standard Time" and "China Standard Time". In this situation, the zone id will be determined by the region information from formatter's locale and the standard zone id for that area, for example, America/New_York for the America Eastern zone. This method also allows a set of preferred ZoneId to be specified for parsing. The matched preferred zone id will be used if the textural zone name being parsed is not unique.

If the zone cannot be parsed then an exception is thrown unless the section of the formatter is optional.

Parameters:

textStyle - the text style to use, not null

preferredZones - the set of preferred zone ids, not null

Returns:

this, for chaining, not null

appendChronologyId

```
public DateTimeFormatterBuilder appendChronologyId()
```

Appends the chronology ID, such as 'ISO' or 'ThaiBuddhist', to the formatter.

This appends an instruction to format/parse the chronology ID to the builder.

During formatting, the chronology is obtained using a mechanism equivalent to querying the temporal with TemporalQueries.chronology(). It will be printed using the result of Chronology.getId(). If the chronology cannot be obtained then an exception is thrown unless the section of the formatter is optional.

During parsing, the chronology is parsed and must match one of the chronologies in Chronology.getAvailableChronologies(). If the chronology cannot be parsed then an exception is thrown unless the section of the formatter is optional. The parser uses the case sensitive setting.

Returns:

this, for chaining, not null

appendChronologyText

```
public DateTimeFormatterBuilder appendChronologyText(TextStyle textStyle)
```

Appends the chronology name to the formatter.

The calendar system name will be output during a format. If the chronology cannot be obtained then an exception will be thrown.

Parameters:

`textStyle - the text style to use, not null`

Returns:

`this, for chaining, not null`

appendLocalized

```
public DateTimeFormatterBuilder appendLocalized(FormatStyle dateStyle,
                                                FormatStyle timeStyle)
```

Appends a localized date-time pattern to the formatter.

This appends a localized section to the builder, suitable for outputting a date, time or date-time combination. The format of the localized section is lazily looked up based on four items:

- the `dateStyle` specified to this method
- the `timeStyle` specified to this method
- the `Locale` of the `DateTimeFormatter`
- the `Chronology`, selecting the best available

During formatting, the chronology is obtained from the temporal object being formatted, which may have been overridden by `DateTimeFormatter.withChronology(Chronology)`.

During parsing, if a chronology has already been parsed, then it is used. Otherwise the default from `DateTimeFormatter.withChronology(Chronology)` is used, with `IsoChronology` as the fallback.

Note that this method provides similar functionality to methods on `DateFormat` such as `DateFormat.getDateTimeInstance(int, int)`.

Parameters:

`dateStyle - the date style to use, null means no date required`

`timeStyle - the time style to use, null means no time required`

Returns:

`this, for chaining, not null`

Throws:

IllegalArgumentException - if both the date and time styles are null

appendLiteral

public DateTimeFormatterBuilder appendLiteral(char literal)

Appends a character literal to the formatter.

This character will be output during a format.

Parameters:

literal - the literal to append, not null

Returns:

this, for chaining, not null

appendLiteral

public DateTimeFormatterBuilder appendLiteral(String literal)

Appends a string literal to the formatter.

This string will be output during a format.

If the literal is empty, nothing is added to the formatter.

Parameters:

literal - the literal to append, not null

Returns:

this, for chaining, not null

append

public DateTimeFormatterBuilder append(DateTimeFormatter formatter)

Appends all the elements of a formatter to the builder.

This method has the same effect as appending each of the constituent parts of the formatter directly to this builder.

Parameters:

formatter - the formatter to add, not null

Returns:

this, for chaining, not null

appendOptional

```
public DateTimeFormatterBuilder appendOptional(DateTimeFormatter formatter)
```

Appends a formatter to the builder which will optionally format/parse.

This method has the same effect as appending each of the constituent parts directly to this builder surrounded by an `optionalStart()` and `optionalEnd()`.

The formatter will format if data is available for all the fields contained within it. The formatter will parse if the string matches, otherwise no error is returned.

Parameters:

`formatter - the formatter to add, not null`

Returns:

`this, for chaining, not null`

appendPattern

```
public DateTimeFormatterBuilder appendPattern(String pattern)
```

Appends the elements defined by the specified pattern to the builder.

All letters 'A' to 'Z' and 'a' to 'z' are reserved as pattern letters. The characters '#', '{' and '}' are reserved for future use. The characters '[' and ']' indicate optional patterns. The following pattern letters are defined:

```
  Symbol  Meaning                     Presentation      Examples
  ------  -------                     ------------      -------
   G       era                         text              AD; Anno Domini; A
   u       year                        year              2004; 04
   y       year-of-era                 year              2004; 04
   D       day-of-year                 number            189
   M/L     month-of-year               number/text       7; 07; Jul; July; J
   d       day-of-month                number            10

   Q/q     quarter-of-year             number/text       3; 03; Q3; 3rd quarter
   Y       week-based-year             year              1996; 96
   w       week-of-week-based-year     number            27
   W       week-of-month               number            4
   E       day-of-week                 text              Tue; Tuesday; T
   e/c     localized day-of-week       number/text       2; 02; Tue; Tuesday; T
   F       week-of-month               number            3

   a       am-pm-of-day                text              PM
   h       clock-hour-of-am-pm (1-12)  number            12
```

```
 K       hour-of-am-pm (0-11)        number           0
 k       clock-hour-of-am-pm (1-24)  number           0

 H       hour-of-day (0-23)          number           0
 m       minute-of-hour              number           30
 s       second-of-minute            number           55
 S       fraction-of-second          fraction         978
 A       milli-of-day                number           1234
 n       nano-of-second              number           987654321
 N       nano-of-day                 number           1234000000

 V       time-zone ID                zone-id          America/Los_Angeles; Z;
 z       time-zone name              zone-name        Pacific Standard Time; P
 O       localized zone-offset       offset-O         GMT+8; GMT+08:00; UTC-08
 X       zone-offset 'Z' for zero    offset-X         Z; -08; -0830; -08:30; -
 x       zone-offset                 offset-x         +0000; -08; -0830; -08:3
 Z       zone-offset                 offset-Z         +0000; -0800; -08:00;

 p       pad next                    pad modifier     1

 '       escape for text             delimiter
 ''      single quote                literal          '
 [       optional section start
 ]       optional section end
 #       reserved for future use
 {       reserved for future use
 }       reserved for future use
```

The count of pattern letters determine the format. See DateTimeFormatter for a user-focused description of the patterns. The following tables define how the pattern letters map to the builder.

Date fields: Pattern letters to output a date.

```
  Pattern  Count  Equivalent builder methods
  -------  -----  -------------------------
    G       1      appendText(ChronoField.ERA, TextStyle.SHORT)
    GG      2      appendText(ChronoField.ERA, TextStyle.SHORT)
    GGG     3      appendText(ChronoField.ERA, TextStyle.SHORT)
    GGGG    4      appendText(ChronoField.ERA, TextStyle.FULL)
    GGGGG   5      appendText(ChronoField.ERA, TextStyle.NARROW)

    u       1      appendValue(ChronoField.YEAR, 1, 19, SignStyle.NORMAL);
    uu      2      appendValueReduced(ChronoField.YEAR, 2, 2000);
    uuu     3      appendValue(ChronoField.YEAR, 3, 19, SignStyle.NORMAL);
    u..u    4..n   appendValue(ChronoField.YEAR, n, 19, SignStyle.EXCEEDS_PAD);
    y       1      appendValue(ChronoField.YEAR_OF_ERA, 1, 19, SignStyle.NORMAL);
    yy      2      appendValueReduced(ChronoField.YEAR_OF_ERA, 2, 2000);
    yyy     3      appendValue(ChronoField.YEAR_OF_ERA, 3, 19, SignStyle.NORMAL);
```

```
y..y    4..n   appendValue(ChronoField.YEAR_OF_ERA, n, 19, SignStyle.EXCEEDS_
Y       1      append special localized WeekFields element for numeric week-b
YY      2      append special localized WeekFields element for reduced numeri
YYY     3      append special localized WeekFields element for numeric week-b
Y..Y    4..n   append special localized WeekFields element for numeric week-b

Q       1      appendValue(IsoFields.QUARTER_OF_YEAR);
QQ      2      appendValue(IsoFields.QUARTER_OF_YEAR, 2);
QQQ     3      appendText(IsoFields.QUARTER_OF_YEAR, TextStyle.SHORT)
QQQQ    4      appendText(IsoFields.QUARTER_OF_YEAR, TextStyle.FULL)
QQQQQ   5      appendText(IsoFields.QUARTER_OF_YEAR, TextStyle.NARROW)
q       1      appendValue(IsoFields.QUARTER_OF_YEAR);
qq      2      appendValue(IsoFields.QUARTER_OF_YEAR, 2);
qqq     3      appendText(IsoFields.QUARTER_OF_YEAR, TextStyle.SHORT_STANDALO
qqqq    4      appendText(IsoFields.QUARTER_OF_YEAR, TextStyle.FULL_STANDALON
qqqqq   5      appendText(IsoFields.QUARTER_OF_YEAR, TextStyle.NARROW_STANDAL

M       1      appendValue(ChronoField.MONTH_OF_YEAR);
MM      2      appendValue(ChronoField.MONTH_OF_YEAR, 2);
MMM     3      appendText(ChronoField.MONTH_OF_YEAR, TextStyle.SHORT)
MMMM    4      appendText(ChronoField.MONTH_OF_YEAR, TextStyle.FULL)
MMMMM   5      appendText(ChronoField.MONTH_OF_YEAR, TextStyle.NARROW)
L       1      appendValue(ChronoField.MONTH_OF_YEAR);
LL      2      appendValue(ChronoField.MONTH_OF_YEAR, 2);
LLL     3      appendText(ChronoField.MONTH_OF_YEAR, TextStyle.SHORT_STANDALO
LLLL    4      appendText(ChronoField.MONTH_OF_YEAR, TextStyle.FULL_STANDALON
LLLLL   5      appendText(ChronoField.MONTH_OF_YEAR, TextStyle.NARROW_STANDAL

w       1      append special localized WeekFields element for numeric week-o
ww      2      append special localized WeekFields element for numeric week-o
W       1      append special localized WeekFields element for numeric week-o
d       1      appendValue(ChronoField.DAY_OF_MONTH)
dd      2      appendValue(ChronoField.DAY_OF_MONTH, 2)
D       1      appendValue(ChronoField.DAY_OF_YEAR)
DD      2      appendValue(ChronoField.DAY_OF_YEAR, 2)
DDD     3      appendValue(ChronoField.DAY_OF_YEAR, 3)
F       1      appendValue(ChronoField.ALIGNED_DAY_OF_WEEK_IN_MONTH)
E       1      appendText(ChronoField.DAY_OF_WEEK, TextStyle.SHORT)
EE      2      appendText(ChronoField.DAY_OF_WEEK, TextStyle.SHORT)
EEE     3      appendText(ChronoField.DAY_OF_WEEK, TextStyle.SHORT)
EEEE    4      appendText(ChronoField.DAY_OF_WEEK, TextStyle.FULL)
EEEEE   5      appendText(ChronoField.DAY_OF_WEEK, TextStyle.NARROW)
e       1      append special localized WeekFields element for numeric day-of
ee      2      append special localized WeekFields element for numeric day-of
eee     3      appendText(ChronoField.DAY_OF_WEEK, TextStyle.SHORT)
eeee    4      appendText(ChronoField.DAY_OF_WEEK, TextStyle.FULL)
eeeee   5      appendText(ChronoField.DAY_OF_WEEK, TextStyle.NARROW)
c       1      append special localized WeekFields element for numeric day-of
ccc     3      appendText(ChronoField.DAY_OF_WEEK, TextStyle.SHORT_STANDALONE
```

```
  cccc    4      appendText(ChronoField.DAY_OF_WEEK, TextStyle.FULL_STANDALONE)
  ccccc   5      appendText(ChronoField.DAY_OF_WEEK, TextStyle.NARROW_STANDALON
```

Time fields: Pattern letters to output a time.

```
  Pattern  Count  Equivalent builder methods
  -------  -----  -------------------------
    a       1      appendText(ChronoField.AMPM_OF_DAY, TextStyle.SHORT)
    h       1      appendValue(ChronoField.CLOCK_HOUR_OF_AMPM)
    hh      2      appendValue(ChronoField.CLOCK_HOUR_OF_AMPM, 2)
    H       1      appendValue(ChronoField.HOUR_OF_DAY)
    HH      2      appendValue(ChronoField.HOUR_OF_DAY, 2)
    k       1      appendValue(ChronoField.CLOCK_HOUR_OF_DAY)
    kk      2      appendValue(ChronoField.CLOCK_HOUR_OF_DAY, 2)
    K       1      appendValue(ChronoField.HOUR_OF_AMPM)
    KK      2      appendValue(ChronoField.HOUR_OF_AMPM, 2)
    m       1      appendValue(ChronoField.MINUTE_OF_HOUR)
    mm      2      appendValue(ChronoField.MINUTE_OF_HOUR, 2)
    s       1      appendValue(ChronoField.SECOND_OF_MINUTE)
    ss      2      appendValue(ChronoField.SECOND_OF_MINUTE, 2)

    S..S    1..n   appendFraction(ChronoField.NANO_OF_SECOND, n, n, false)
    A       1      appendValue(ChronoField.MILLI_OF_DAY)
    A..A    2..n   appendValue(ChronoField.MILLI_OF_DAY, n)
    n       1      appendValue(ChronoField.NANO_OF_SECOND)
    n..n    2..n   appendValue(ChronoField.NANO_OF_SECOND, n)
    N       1      appendValue(ChronoField.NANO_OF_DAY)
    N..N    2..n   appendValue(ChronoField.NANO_OF_DAY, n)
```

Zone ID: Pattern letters to output `ZoneId`.

```
  Pattern  Count  Equivalent builder methods
  -------  -----  -------------------------
    VV      2      appendZoneId()
    z       1      appendZoneText(TextStyle.SHORT)
    zz      2      appendZoneText(TextStyle.SHORT)
    zzz     3      appendZoneText(TextStyle.SHORT)
    zzzz    4      appendZoneText(TextStyle.FULL)
```

Zone offset: Pattern letters to output `ZoneOffset`.

```
  Pattern  Count  Equivalent builder methods
  -------  -----  -------------------------
    O       1      appendLocalizedOffsetPrefixed(TextStyle.SHORT);
    OOOO    4      appendLocalizedOffsetPrefixed(TextStyle.FULL);
    X       1      appendOffset("+HHmm","Z")
    XX      2      appendOffset("+HHMM","Z")
    XXX     3      appendOffset("+HH:MM","Z")
```

```
  XXXX    4      appendOffset("+HHMMss","Z")
  XXXXX   5      appendOffset("+HH:MM:ss","Z")
  x       1      appendOffset("+HHmm","+00")
  xx      2      appendOffset("+HHMM","+0000")
  xxx     3      appendOffset("+HH:MM","+00:00")
  xxxx    4      appendOffset("+HHMMss","+0000")
  xxxxx   5      appendOffset("+HH:MM:ss","+00:00")
  Z       1      appendOffset("+HHMM","+0000")
  ZZ      2      appendOffset("+HHMM","+0000")
  ZZZ     3      appendOffset("+HHMM","+0000")
  ZZZZ    4      appendLocalizedOffset(TextStyle.FULL);
  ZZZZZ   5      appendOffset("+HH:MM:ss","Z")
```

Modifiers: Pattern letters that modify the rest of the pattern:

```
  Pattern  Count  Equivalent builder methods
  -------  -----  -------------------------
    [       1      optionalStart()
    ]       1      optionalEnd()
    p..p    1..n   padNext(n)
```

Any sequence of letters not specified above, unrecognized letter or reserved character will throw an exception. Future versions may add to the set of patterns. It is recommended to use single quotes around all characters that you want to output directly to ensure that future changes do not break your application.

Note that the pattern string is similar, but not identical, to `SimpleDateFormat`. The pattern string is also similar, but not identical, to that defined by the Unicode Common Locale Data Repository (CLDR/LDML). Pattern
letters 'X' and 'u' are aligned with Unicode CLDR/LDML. By contrast, `SimpleDateFormat` uses 'u' for the numeric day of week. Pattern letters 'y' and 'Y' parse years of two digits and more than 4 digits differently. Pattern letters 'n', 'A', 'N', and 'p' are added. Number types will reject large numbers.

Parameters:

`pattern - the pattern to add, not null`

Returns:

`this, for chaining, not null`

Throws:

`IllegalArgumentException - if the pattern is invalid`

padNext

```
public DateTimeFormatterBuilder padNext(int padWidth)
```

Causes the next added printer/parser to pad to a fixed width using a space.

This padding will pad to a fixed width using spaces.

During formatting, the decorated element will be output and then padded to the specified width. An exception will be thrown during formatting if the pad width is exceeded.

During parsing, the padding and decorated element are parsed. If parsing is lenient, then the pad width is treated as a maximum. The padding is parsed greedily. Thus, if the decorated element starts with the pad character, it will not be parsed.

Parameters:

`padWidth - the pad width, 1 or greater`

Returns:

`this, for chaining, not null`

Throws:

`IllegalArgumentException - if pad width is too small`

padNext

```
public DateTimeFormatterBuilder padNext(int padWidth,
                                        char padChar)
```

Causes the next added printer/parser to pad to a fixed width.

This padding is intended for padding other than zero-padding. Zero-padding should be achieved using the appendValue methods.

During formatting, the decorated element will be output and then padded to the specified width. An exception will be thrown during formatting if the pad width is exceeded.

During parsing, the padding and decorated element are parsed. If parsing is lenient, then the pad width is treated as a maximum. If parsing is case insensitive, then the pad character is matched ignoring case. The padding is parsed greedily. Thus, if the decorated element starts with the pad character, it will not be parsed.

Parameters:

`padWidth - the pad width, 1 or greater`

`padChar - the pad character`

Returns:

`this, for chaining, not null`

Throws:

`IllegalArgumentException` - `if pad width is too small`

optionalStart

```
public DateTimeFormatterBuilder optionalStart()
```

Mark the start of an optional section.

The output of formatting can include optional sections, which may be nested. An optional section is started by calling this method and ended by calling `optionalEnd()` or by ending the build process.

All elements in the optional section are treated as optional. During formatting, the section is only output if data is available in the `TemporalAccessor` for all the elements in the section. During parsing, the whole section may be missing from the parsed string.

For example, consider a builder setup as `builder.appendValue(HOUR_OF_DAY, 2).optionalStart().appendValue(MINUTE_OF_HOUR,2)`. The optional section ends automatically at the end of the builder. During formatting, the minute will only be output if its value can be obtained from the date-time. During parsing, the input will be successfully parsed whether the minute is present or not.

Returns:

`this, for chaining, not null`

optionalEnd

```
public DateTimeFormatterBuilder optionalEnd()
```

Ends an optional section.

The output of formatting can include optional sections, which may be nested. An optional section is started by calling `optionalStart()` and ended using this method (or at the end of the builder).

Calling this method without having previously called `optionalStart` will throw an exception. Calling this method immediately after calling `optionalStart` has no effect on the formatter other than ending the (empty) optional section.

All elements in the optional section are treated as optional. During formatting, the section is only output if data is available in the

TemporalAccessor for all the elements in the section. During parsing, the whole section may be missing from the parsed string.

For example, consider a builder setup as builder.appendValue(HOUR_OF_DAY, 2).optionalStart().appendValue(MINUTE_OF_HOUR,2).optionalEnd(). During formatting, the minute will only be output if its value can be obtained from the date-time. During parsing, the input will be successfully parsed whether the minute is present or not.

Returns:

this, for chaining, not null

Throws:

IllegalStateException - if there was no previous call to optionalStart

toFormatter

public DateTimeFormatter toFormatter()

Completes this builder by creating the DateTimeFormatter using the default locale.

This will create a formatter with the default FORMAT locale. Numbers will be printed and parsed using the standard DecimalStyle. The resolver style will be SMART.

Calling this method will end any open optional sections by repeatedly calling optionalEnd() before creating the formatter.

This builder can still be used after creating the formatter if desired, although the state may have been changed by calls to optionalEnd.

Returns:

the created formatter, not null

toFormatter

public DateTimeFormatter toFormatter(Locale locale)

Completes this builder by creating the DateTimeFormatter using the specified locale.

This will create a formatter with the specified locale. Numbers will be printed and parsed using the standard DecimalStyle. The resolver style will be SMART.

Calling this method will end any open optional sections by repeatedly calling `optionalEnd()` before creating the formatter.

This builder can still be used after creating the formatter if desired, although the state may have been changed by calls to `optionalEnd`.

Parameters:

`locale - the locale to use for formatting, not null`

Returns:

`the created formatter, not null`

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.